

THE PLAN

YourKalinga

WC Launchpad Builder Round · Telehealth App

Software Engineering Track · May 26 – May 30, 2026 · 5-Day Build Window

VERSION 4 · 100% Free-Tier & Open-Source Stack

Product identity — YourKalinga. From the Filipino word *kalinga* (“care” / “to tend to someone”), prefixed with **Your** to signal personal, patient-owned care. The name sets the product tone: warm, trustworthy, and Filipino-rooted — directly supporting the trust-first design language (Section 03) and the Tagalog accessibility play (Section 09). Every piece of microcopy should feel like personal care, not a transactional clinic.

This document contains:

① Requirement Breakdown (verbatim from spec) ② Fast-Ship Engineering Practices ③ Telehealth Market Insights ④ Feature Priority Map ⑤ Technical Expectations Alignment ⑥ Recommended Tech Stack (free-tier) ⑦ File Architecture ⑧ 5-Day Sprint Plan ⑨ Bonus Differentiators ⑩ Env Variables Reference

Stack guarantee: every component is either open source (self-hosted free) or runs on a permanent free tier with no credit card required. Total infrastructure cost to build, deploy, and demo YourKalinga: **\$0/month**. See Section 06 for the full cost breakdown.

Scoring Criteria — Software Engineering Track

Competency	Evaluation Guide	SE Weight
Functionality & Scope Covered	How closely were the requirements implemented as described?	40%
Design & Product Sense	How well was the app designed? Good product sense shown?	40%
Adherence & Code Quality	How closely were the technical expectations followed?	10%
Presentation & Communication	How well was the video presentation delivered?	10%

01 · Requirement Breakdown

These tables are **directly derived from requirements.pdf** with no detail dropped. Every item is marked **Required** and must ship in **YourKalinga** before any bonus work. Two roles: **Patient** and **Doctor**.

Patient Module

The following are patient-facing features:

Feature	Description	Required Details	Req.
Patient Account Creation	Allows patients to securely register and create their own YourKalinga account. The entry point to the platform.	• Register using email and password • Input profile: Name, Birthday, Weight, Height, Profile Picture, Contact Details, Basic Medical History	Required
Doctor Discovery	Enables patients to browse available doctors and review their schedules before booking.	• Browse doctors and view their availability • Explore doctors based on medical needs/symptoms • Filter/search doctors by specialization	Required
AI Recommendation	Allows patients to describe symptoms or healthcare concerns and receive suggested doctors based on specialization or expertise.	• AI recommends doctor based on patient needs	Required
Appointment Booking	Allows patients to schedule online consultations with doctors based on available schedules.	• Book consultations online • Reschedule or cancel schedules • Real-time push notifications: notify users for booked, upcoming appointments, and schedule updates	Required
Consultation Session	Provides patients access to a virtual consultation experience to join their scheduled appointment online. <i>Note: does NOT require a fully custom-built video conferencing solution.</i>	• Join a consultation session	Required
Medical Records	Allows patients to review previous consultations and view records provided by doctors after each session.	• View appointment history • View basic medical records or prescriptions	Required

Doctor Module

The following are doctor / admin-facing features:

Feature	Description	Required Details	Req.
Doctor Profile Management	Allows doctors to securely register and create their own account.	• Register using email and password • Add profile details, bio, and specialization	Required

Medical Records Access	Enables doctors to review patient consultation history and previously issued records or prescriptions.	• View appointment history and medical records/prescriptions	Required
Consultation Schedule Management	Allows doctors to manage consultation availability and ensure schedules are properly organized.	• Manage consultation schedules • Restrict unavailable time slots • Real-time push notifications for booked, upcoming appointments, and schedule updates	Required
Consultation Notes & Prescriptions	Allows doctors to document findings, recommendations, prescriptions, and consultation summaries after each appointment.	• Add prescriptions and/or medical consultation notes	Required
Consultation Session	Enables doctors to join and conduct virtual consultations with patients. <i>Note: does NOT require a fully custom-built video conferencing solution.</i>	• Join consultation session	Required

Video Consultation (spec verbatim): the consultation session does NOT require a fully custom-built video conferencing solution. YourKalinga uses **Jitsi Meet** — fully open source (Apache-2.0), free, embeddable via iframe with zero API key on the public meet.jit.si instance, or self-hostable in Docker.

02 · Fast-Ship Engineering Practices

Shipping a production-quality MVP in 5 days requires discipline over creativity. These practices are non-negotiable throughout the YourKalinga build.

1	Vertical Slices, Not Horizontal Layers Build one complete journey end-to-end (Register → Book → Join) on Day 1 before touching other features. Proves the full stack works and leaves you demo-able even if time runs out.
2	Database-First Schema Design Lock your Prisma schema in the first 2 hours. ALL API and UI work derives from it. Schema changes mid-sprint cascade everywhere — avoid at all costs.
3	API Contract First Write a short markdown table of endpoints + payloads before coding. Frontend and backend then work in parallel — the frontend mocks the API while the backend builds it.
4	Component-Driven UI Build small reusable components first (Button, Card, Badge, Avatar, Modal). Every page is assembled from these. Shadcn/UI skips ~80% of component work.
5	Seed Data on Day 1 Generate 8–10 fake doctors with specializations, bios, schedules, and time slots via a seed script immediately. You cannot test Discovery or Booking without data.
6	Feature Flags from Day 0 Wrap incomplete features behind a boolean env var. Ship safely without comment-bombing half-finished code — evaluators see a clean codebase.
7	Environment Parity Run Docker Compose locally from Day 1 so dev mirrors production. Eliminates 'works on my machine' surprises at deployment time.
8	Trunk-Based Development Work on main with short-lived branches under 4 hours each. Push frequently. Avoid long-lived branches that create merge conflicts under time pressure.
9	AI-Assisted Development Use Claude/ChatGPT for boilerplate, tests, debugging, JSDoc, and seed data. Cursor and Windsurf are explicitly encouraged by the spec. Never hand-write repetitive code.
10	Fail Fast with Typed Error Handling Wrap every async operation in try/catch with typed error responses. Surface errors immediately. Code quality is 10% of the score and evaluators read your error patterns.

03 · Telehealth Market Insights

Understanding what makes leading platforms sticky is critical for **Design & Product Sense** (40%). Analysis draws from Teladoc, MDLive, Doximity, Doctor on Demand, and Amwell.

► Trust Signals at Every Step

Doctor credentials, certifications, and photo-first profiles are front and center. Patients hand over health data — they must feel safe first. Use verified badges, clear privacy messaging, and professional visual language.

► Effortless Onboarding (Reduce Friction)

Teladoc and MDLive minimize form fields. Patients answer only what's needed to book a first consultation. Defer non-essential profile data to post-registration nudges. Every extra field is a drop-off risk.

► One-Tap Access to Care

The most successful apps prioritize speed-to-consultation. 'See a doctor now' is always 1-2 taps from any screen. Target booking in under 3 steps.

► Post-Consultation Retention Loop

The 48 hours after a visit decide retention. A follow-up summary, prescription confirmation, and 1-tap rebooking prompt drive repeat visits. Doctor on Demand and Teladoc both use this.

► Humanized Doctor Profiles

Doximity sets the benchmark: bios, languages spoken, photo, real-time availability, and patient reviews. Patients book people, not time slots.

► Proactive Notifications Done Right

Reminders at 24h and 1h before, instant booking confirmations, and prescription-ready alerts keep users engaged without requiring app opens.

► Clarity Over Complexity

Healthcare users are often stressed or unwell. Successful UX uses large tap targets, plain language, familiar icons (clipboard = records, pill = medication), and a calm palette (teal, blue, white).

Design Language Takeaways for YourKalinga

- **Primary palette: teal + white** — conveys cleanliness, trust, and calm.
- **Whitespace over density** — less clutter equals more perceived trust.
- **Real-time doctor availability** — never show a doctor whose slots are all booked.
- **Skeleton loaders over spinners** — feels faster and more polished.
- **Empty states with clear CTAs** — e.g. 'No appointments yet — book your first consultation'.
- **Confirm all destructive actions** — cancelling an appointment needs a confirmation modal.
- **Mobile-responsive from the start** — spec requires desktop-oriented but responsive.
- **Brand voice = warmth + ownership** — “YourKalinga” means *your* care; write microcopy that reassures (“We’ve got you”), never clinical jargon. Lean on the Filipino-care identity as the differentiator generic competitors can’t copy.

04 · Feature Priority Map

All features are P0 (Required). The order below is the correct build sequence based on dependency chains. **Do not deviate from this order.**

⚠ Build order: Database Schema (#2) must be built BEFORE the Auth module (#3). You cannot write auth logic against a schema that doesn't exist. Correct chain: Stack setup → DB Schema → Auth → everything else.

#	Feature	Est. Time	Depends On	Risk
1	Project Setup + Docker Compose + Monorepo	1.5h	—	HIGH
2	Database Schema + Prisma Migrations + Seed Script	2.5h	Stack setup	HIGH
3	Auth Module — Register/Login (both roles, JWT)	3h	DB schema (#2)	HIGH
4	Doctor Profile CRUD (API + frontend form)	2h	Auth (#3)	MED
5	Patient Profile CRUD (API + frontend form)	2h	Auth (#3)	MED
6	Doctor Discovery: browse, search, filter by specialization	3h	Doctor Profile (#4)	MED
7	AI Symptom Checker → ranked doctor recommendations	2.5h	Profiles (#4), Groq API	MED
8	Doctor Schedule Management (set + restrict slots)	2.5h	Doctor Profile (#4)	MED
9	Appointment Booking — patient books a slot	4h	Schedules (#8), Profiles	HIGH
10	Reschedule / Cancel Appointment	2h	Booking (#9)	MED
11	Real-time Push Notifications (Socket.IO)	2.5h	Booking (#9) + Schedules (#8)	MED
12	Consultation Session page (Jitsi embed)	2h	Booking (#9)	LOW
13	Consultation Notes + Prescriptions (doctor post-session)	2h	Session (#12)	MED
14	Medical Records view — history + prescriptions	2h	Notes (#13), Booking	LOW

Total estimated build time: ~33 hours. Across 5 days (May 26–30) at ~7–8h/day, this leaves real time for UI polish, bonus features, deployment hardening, and recording the video. Do not start bonus features until all 14 tasks work on the deployed URL.

05 · Technical Expectations (from requirements.pdf)

This section reproduces the **exact** technical constraints from the spec. Every choice in Section 06 is validated against these.

Category	Allowed Options	Notes / Constraints
Platform	• Web only • Desktop-oriented (responsive design)	No mobile-native app. Must work on desktop browsers first, adapt down to mobile.
Frontend	• Next.js • Flutter • React	Prefer strongly-typed language (TypeScript, Dart). OK to use a UI framework: Shadcn, Radix, etc.
Backend	• NestJS • Node.js • SpringBoot	Avoid BaaS (Firebase, Supabase) for databases. Prefer custom APIs for most backend logic. BaaS allowed selectively (Auth, Chat) — not as primary backend.
Database	Any persistent DB: • MongoDB • PostgreSQL • MySQL • MariaDB / etc.	Must be a real persistent database — not an in-memory store.
Version Control	• GitHub • GitLab • Bitbucket	Public repository link required for submission.
Code Quality	• Modular • Well-documented • With error handling	10% of score. Clean modules, JSDoc comments, typed errors.
Deployment	• Terraform / Docker • Vercel / Fly.io • Heroku / AWS • CloudFlare	Prefer containerized deployment. Application MUST be accessible via a public URL.
Others	• AI assistants: Cursor, Windsurf (encouraged) • Vibe tools: Lovable, V0 (allowed) • Third-party libs: allowed	You must be able to explain your implementation and architecture even if AI tools were used.

Official Tips from requirements.pdf

Verbatim from the spec:

- We value thoughtful product decisions, clean architecture, strong execution, and polished user experiences over excessive feature quantity.
- A smaller but well-designed and complete solution is preferred over an overly ambitious but unfinished implementation.
- Thoughtful UX decisions, prioritization, and overall product sense will be heavily considered during evaluation.
- Focus on the core functionalities first before moving on to bonus or experimental features.

06 · Recommended Tech Stack (Free-Tier & Open-Source)

Every choice is validated against Section 05 **and** guaranteed to be free. The **Tier** column shows whether each item is open source (OSS, self-hosted free) or a managed service used on its permanent free tier. Nothing in the YourKalinga stack requires a paid subscription or a credit card.

Layer	Choice	Tier	Rationale
Frontend	Next.js 14 + TypeScript	OSS · Free	MIT-licensed. In spec. TypeScript satisfies the strongly-typed preference. App Router + best AI codegen support.
UI Framework	Shadcn/UI + Tailwind CSS	OSS · Free	MIT. Spec allows Shadcn/Radix. Copy-paste primitives, accessible, polished. Saves ~2 days of component work.
State	Zustand + TanStack Query	OSS · Free	MIT. Zustand for global auth/user state; TanStack Query for server state, caching, background refetch.
Backend	NestJS + TypeScript	OSS · Free	MIT. In spec. Modular, decorators, DI. Enforces clean, testable code (Code Quality criterion).
Database	PostgreSQL + Prisma ORM	OSS · Free	Both open source. In spec. Relational model fits users/appointments/records. Prisma generates shared types.
DB Hosting	Neon (free tier)	Free tier	Managed Postgres, NOT a BaaS — stays within spec. Free tier: ~0.5 GB storage, autosuspend. No card.
Real-time	Socket.IO (NestJS Gateway)	OSS · Free	MIT. WebSocket notifications for bookings and schedule updates. Same port as REST, single container.
Video	Jitsi Meet	OSS · Free	Apache-2.0. Custom WebRTC NOT required by spec. Embed meet.jit.si via iframe (no key) or self-host in Docker.
Auth	Custom JWT + bcrypt	OSS · Free	MIT libs. Spec prefers custom APIs. JWT + bcrypt in NestJS Guards. Fully self-contained, no external service.
AI Feature	Groq API (free tier)	Free tier	Free, no card. OpenAI-compatible. Runs open-weight Llama 3.3 70B. Powers AI Recommendation with JSON output.
File Storage	Cloudinary (free tier)	Free tier	Free tier for profile pics + attachments. Fully-OSS alternative: MinIO (S3-compatible) in Docker Compose.
Containerization	Docker + Docker Compose	OSS · Free	Apache-2.0. Spec prefers containers. One-command local dev (web + api + postgres).
Backend Deploy	Render (free tier)	Free tier	Free Docker web service, public URL, no card. Note: spins down after 15 min idle (30–50s cold start).
Frontend Deploy	Vercel (free hobby)	Free tier	Free hobby tier, purpose-built for Next.js. In spec. Instant deploys from GitHub, always-on, no cold start.
Version Control	GitHub (free)	Free tier	Required by spec. Free public/private repos. Public repo link for submission; powers Render/Vercel CI/CD.

Total infrastructure cost: \$0/month. 11 components are open source (self-hosted free); 4 are managed services on permanent free tiers (Neon, Groq, Cloudinary, Vercel) plus Render free hosting — none require a credit card. YourKalinga can be built, deployed to a public URL, and demoed for free.

⚠ Two free-tier caveats to plan around: (1) **Render cold starts** — the free backend sleeps after 15 min idle and takes 30–50s to wake. Open your public URL ~1 minute before recording the demo video so it's warm. (2) **Groq rate limits** — roughly 30 requests/min and ~1,000–14,400 requests/day on the free tier. Far more than a demo needs, but cache AI results per symptom query to be safe.

07 · Project File Architecture

A monorepo keeps frontend and backend in one GitHub repository while remaining independently deployable. Clean to navigate during evaluation and efficient for AI assistant context windows.

Repository Structure

yourkalinga/	Monorepo root (GitHub: yourkalinga)
├─ apps/	
│ └─ web/	Next.js 14 Frontend → deploy to Vercel
│ └─ app/	App Router pages
│ └─ (auth)/	Login, Register (both roles)
│ └─ (patient)/	Patient-facing pages
│ └─ dashboard/	
│ └─ doctors/	Discovery + AI Recommend
│ └─ appointments/	Book, Reschedule, Cancel
│ └─ consultation/	Jitsi video session room
│ └─ records/	Medical history + prescriptions
│ └─ (doctor)/	Doctor-facing pages
│ └─ dashboard/	
│ └─ schedule/	Manage availability, restrict slots
│ └─ appointments/	Upcoming + history
│ └─ consultation/	Jitsi session + notes/Rx form
│ └─ patients/	Patient records access
│ └─ api/	Next.js API routes (thin proxy)
│ └─ components/	
│ └─ ui/	Shadcn primitives
│ └─ shared/	Navbar, Sidebar, NotificationDrawer
│ └─ patient/	Patient-specific components
│ └─ doctor/	Doctor-specific components
│ └─ lib/ (api.ts, auth.ts, socket.ts, jitsi.ts)	
│ └─ hooks/	Custom React hooks (useAuth, useNotif)
│ └─ store/	Zustand stores (auth, notifications)
│ └─ types/	Shared TypeScript interfaces
├─ api/	NestJS Backend → deploy to Render
│ └─ src/	
│ └─ auth/	JWT strategy, guards, refresh tokens
│ └─ users/	Base user entity + service
│ └─ doctors/	Profile + schedule + availability
│ └─ patients/	Patient profile + medical history
│ └─ appointments/	Booking, reschedule, cancel logic
│ └─ consultations/	Session, notes, prescriptions
│ └─ medical-records/	Records access (doctor + patient)
│ └─ notifications/	Socket.IO WebSocket gateway
│ └─ ai/	Groq API: symptom → doctor suggest
│ └─ uploads/	Cloudinary service (profile pics)
│ └─ common/	DTOs, decorators, filters, pipes
│ └─ main.ts	
│ └─ prisma/	
│ └─ schema.prisma	DB schema → Neon Postgres (db: yourkalinga)
│ └─ seed.ts	8–10 fake doctors + time slots
├─ docker-compose.yml	Local: web + api + postgres
├─ docker-compose.prod.yml	Production build (Render) overrides
├─ .env.example	All env vars (no secrets)
└─ README.md	Setup, architecture, deployment

Key Architecture Decisions

- **Role-based routing:** Patient and Doctor pages live in separate Next.js route groups with independent middleware guards.

- **Prisma as single source of truth:** the schema generates TypeScript types used by both frontend and backend — no type drift.
- **Thin API proxy:** Next.js /api routes forward to NestJS — keeps CORS simple and allows independent scaling later.
- **Socket.IO gateway:** real-time notifications run on the same port as REST — one Docker container, no extra service.
- **AI module isolation:** Groq calls are encapsulated in src/ai/ via the OpenAI-compatible SDK — swappable to any provider by changing the base URL.
- **Split free deploy:** Vercel hosts the always-on YourKalinga frontend; Render hosts the Dockerized NestJS API; Neon hosts Postgres — three free tiers, one public app.

08 · 5-Day Sprint Plan (May 26 - May 30)

Each day ends with a clear, demo-able milestone. Never end a day without something working. The 5-day window matches the official program timeline from requirements.pdf.

DAY 1 · May 26 · Foundation

[0.0–1.0h] Create the 'yourkalinga' GitHub monorepo. Docker Compose (web + api + postgres) for local dev.

[1.0–2.5h] Prisma schema: User, DoctorProfile, PatientProfile, Schedule, Appointment, ConsultationNote, MedicalRecord, Notification.

[2.5–4.0h] Run migrations. Seed script: 8–10 doctors, specializations, bios, time slots.

[4.0–6.0h] Auth module: JWT register/login both roles, guards, refresh tokens.

[6.0–8.0h] Doctor Profile CRUD + form. Patient Profile CRUD + form.

[8.0–9.0h] Doctor Discovery: card list, search bar, filter by specialization.

[9.0–10.h] Navigation shell: branded YourKalinga Navbar, Sidebar, role-based routing.

✓ **MILESTONE: both roles register, log in, edit profiles. Doctors are discoverable.**

DAY 2 · May 27 · Booking Core

[0.0–2.0h] Doctor Schedule Management: set available slots, restrict unavailable slots.

[2.0–5.0h] Appointment Booking: patient picks doctor → views slots → selects time → confirms.

[5.0–6.5h] Reschedule and Cancel flows with confirmation modals.

[6.5–8.5h] Socket.IO gateway: booking-confirmation event + upcoming-reminder logic.

[8.5–10.h] Notification UI: bell icon, unread badge, notification drawer.

✓ **MILESTONE: patient books, reschedules, cancels. Doctor receives booking notifications.**

DAY 3 · May 28 · AI + Consultation

[0.0–2.5h] AI Symptom Checker: input → Groq API (Llama 3.3 70B) → ranked doctor suggestions on discovery page.

[2.5–4.0h] Consultation Session page: embed Jitsi Meet. Join active only within appointment window.

[4.0–5.5h] Consultation Notes + Prescriptions: doctor post-session form linked to the appointment.

[5.5–7.0h] Patient Medical Records: history, notes, and prescriptions from past consultations.

[7.0–10.h] Doctor Medical Records Access: view patient history + issued records.

✓ **MILESTONE: AI recommendation works. Full loop: join → note → view record.**

DAY 4 · May 29 · Polish + Deployment

[0.0–2.0h] UI polish: skeleton loading, empty states with CTAs, error messages, validation feedback.

[2.0–3.5h] Mobile responsiveness pass: all pages usable on tablet/mobile.

[3.5–5.0h] Code quality: JSDoc, typed DTOs, exception filters, remove console.logs.

[5.0–6.5h] Deploy: Neon (Postgres) → Render (NestJS API, Docker) → Vercel (Next.js). Wire env vars. Test public URL.

[6.5–8.5h] End-to-end smoke test on production: full patient + doctor journey.

[8.5–10.h] Fix production-only bugs (CORS, env, cold start). Write README with setup instructions.

✓ **MILESTONE: YourKalinga fully deployed at a public URL, \$0 infrastructure. All P0 features working.**

DAY 5 · May 30 · Bonus + Presentation

[0.0–3.0h] Implement 1–2 bonus features (Section 09). Start with AI Pre-Consultation Intake or Doctor Ratings.

[3.0–4.0h] Final production deploy with bonus features. Smoke test again.

[4.0–6.5h] Warm the Render URL, then record video (Loom/OBS): Problem → Solution → Patient demo → Doctor demo → Architecture → Bonus.

[6.5–7.5h] Review video: speak to design decisions (incl. the YourKalinga name story), not just feature clicks.

[7.5–10.h] Buffer: last-minute bug fixes, finalize README, submit.

✓ **MILESTONE: submission-ready. Public URL live. Video recorded. Repo clean.**

09 · Bonus Features & Differentiators

The spec asks two questions for bonus features. Each feature below maps to one or both. All listed options stay within the YourKalinga free-tier guarantee.

Bonus Q1: How does YourKalinga differentiate itself from other similar platforms?

Bonus Q2: How does it elevate the patient and doctor journey to create a seamless, intuitive, engaging experience that encourages long-term user retention?

Bonus Feature	Effort	Answers	Description
AI Pre-Consultation Intake	MED	Q1+Q2	Before the appointment, prompt the patient with 5 structured health questions (chief complaint, duration, severity, medications, allergies). Summarize the intake (via Groq) into the doctor's dashboard before the session. Doctors save time; patients feel heard.
Patient Health Timeline	MED	Q2	Visualize consultation history as a chronological timeline: prescriptions, vitals changes (if logged), and upcoming appointments. Gives patients a clear view of their health journey — strong retention driver.
Doctor Rating and Reviews	LOW	Q1+Q2	After a completed consultation, prompt for a 1-5 star rating and optional comment. Show aggregated rating on each Discovery card. Builds trust like Teladoc/MDLive.
In-App Chat (Pre-Consultation)	MED	Q2	Let patients message their booked doctor before the session. Build it free with your existing Socket.IO gateway (no paid chat SaaS needed) — a new messages channel reuses the WebSocket infrastructure.
Multi-Language — Filipino (Tagalog)	LOW	Q1+Q2	Add Tagalog via next-intl (open source). Reinforces the YourKalinga brand identity and boosts accessibility for the Filipino market — a differentiator generic competitors lack.
Google Calendar / ICS Export	LOW	Q2	'Add to Calendar' button on confirmed bookings generates a downloadable .ics file for Google/Apple/Outlook. Pure open-source generation, no API. Simple to build, high perceived value.

Presentation Tips (10% of score)

- **Record on the live deployed URL** — never a localhost demo. Warm the Render URL first to avoid the cold-start delay.
- **Open with the name story** — “YourKalinga: *kalinga* is Filipino for care” instantly signals empathy and product sense.
- **Structure:** Problem → Solution → Patient demo → Doctor demo → Architecture (30s) → Bonus features.
- **Speak to design decisions**, not just clicks. 'I chose teal because trust matters in healthcare' scores higher.
- **Mention the free-tier architecture** — building a full-stack telehealth MVP at \$0 infra cost is itself a strong engineering signal.
- **Keep it concise** — under 5 minutes unless told otherwise. Evaluators review many submissions.

- **Lead with the AI Recommendation feature** — the clearest differentiator and proof of product sense.

10 · Environment Variables Reference

Copy into **.env.example** on Day 1, add **.env** to **.gitignore** immediately, fill values as you go. Set production values via the Render / Vercel / Neon dashboards. Every key below is for a free-tier service.

```
# — Database (Neon free tier – managed Postgres) ———
DATABASE_URL=postgresql://user:password@ep-xxx.neon.tech/yourkalinga?sslmode=require

# — Auth (custom JWT – no external service) ———
JWT_SECRET=your-super-secret-jwt-key-change-this
JWT_EXPIRES_IN=7d
JWT_REFRESH_EXPIRES_IN=30d

# — Groq (AI Recommendation – free tier, no card) ———
GROQ_API_KEY=gsk_...
GROQ_MODEL=llama-3.3-70b-versatile
GROQ_BASE_URL=https://api.groq.com/openai/v1

# — Video (Jitsi Meet – open source, free) ———
# Public instance needs no key – just embed a room URL:
NEXT_PUBLIC_JITSI_DOMAIN=meet.jit.si
NEXT_PUBLIC_JITSI_ROOM_PREFIX=yourkalinga-
# (If self-hosting Jitsi in Docker, point JITSI_DOMAIN at your domain instead)

# — File Storage (Cloudinary free tier) ———
CLOUDINARY_CLOUD_NAME=your-cloud-name
CLOUDINARY_API_KEY=your-api-key
CLOUDINARY_API_SECRET=your-api-secret
# (Fully-OSS alternative: run MinIO in docker-compose and use S3 vars)

# — App ———
NEXT_PUBLIC_APP_NAME=YourKalinga
NEXT_PUBLIC_API_URL=http://localhost:3001
NEXT_PUBLIC_WS_URL=ws://localhost:3001
PORT=3001
NODE_ENV=development
FRONTEND_URL=http://localhost:3000
# Production: set API_URL to the Render URL, FRONTEND_URL to the Vercel URL
```

⚠ NEVER commit **.env** to GitHub. Add it to **.gitignore** on Day 1 before your first commit. Commit only **.env.example** with placeholders. On Render/Vercel/Neon, set all secrets through the platform dashboard — never in code or **docker-compose.yml** directly. The Groq key starts with 'gsk_' and the Neon connection string requires '?sslmode=require'.